Solution Approach: Frankenstein

Meta-Agentic System for Autonomous Agent Construction

1. Core Premise

Building an agent requires two kinds of knowledge: **domain knowledge** (what the agent should do) and **engineering knowledge** (how to build it). Today, one person needs both. Frankenstein splits that burden — the human provides domain expertise, Frankenstein provides the engineering.

The human describes a problem in plain language. Frankenstein asks the right questions to extract what it needs, designs the architecture, stress-tests it, builds it, runs it, and fixes what breaks. The human validates twice — after requirements and after the architectural blueprint — then Frankenstein handles the rest autonomously.

2. Architecture

```
Human                          Frankenstein
______                         ____________


Fuzzy prompt ──────────────►  [1. ELICITOR AGENT]
                               Structured domain extraction
                               5 categories of targeted questions
                                      │
                                      ▼
                               Requirements Document
                                      │
Human validates ◄──────────── "Is this what you meant?"
_______________                       │
                                      ▼
                              [2. ARCHITECT AGENT]
                               Generates framework-agnostic spec
                               Selects tools from validated library
                               Designs agent roles, memory, flow
                                      │
                                      ▼
```

```
                                [3. CRITIC AGENT]
                                 Multi-vector attack on spec
                                 Edge cases, dead-ends, tool mismatches
                                 Architect revises until no criticals
                                        |
                                        ▼
                                 Validated Spec + Attack Report
                                        |
Human reviews ◄──────────────── "Here's the blueprint.
──────────────                   Here's what could go wrong.
                                 Approve?"
                                        |
                                        ▼
                                [4. BUILDER AGENT]
                                 Compiles spec → CrewAI / LangGraph code
                                 Template-driven, not free-form generation
                                        |
                                        ▼
                                [5. TESTER AGENT]
                                 Runs agents in Docker sandbox
                                 Validates output against spec contracts
                                 Traces failures → spec-level root cause
                                        |
                                        ▼
                                 Working, tested agents
                                 + Learnings → Chroma (for future builds)
```

Why This Pipeline, Not One LLM Call

A single LLM prompt can generate agent code. But it cannot:

- Ask the human what's missing from their requirements
- Validate that selected tools actually support the required data formats
- Attack its own design for failure modes before building
- Run the generated code and trace failures back to architectural decisions
- Learn from previous builds to improve future ones

Each stage exists because it solves a specific failure mode that one-shot generation cannot.

3. Pipeline Orchestration

Frankenstein itself is a LangGraph `StateGraph`. Each stage is a node. Edges define transitions. Conditional edges implement feedback loops.

3.1 Pipeline State Object

A single state object flows through the entire graph. Every node reads from it, does its work, writes back:

```
class FrankensteinState(TypedDict):
    # Stage 1: Elicitor
    raw_prompt: str
    elicitor_questions: list[dict]         # generated questions per category
    human_answers: list[dict]              # human responses
    requirements: RequirementsDoc          # structured output
    requirements_approved: bool            # human checkpoint 1

    # Stage 2-3: Architect + Critic
    tool_library_matches: list[ToolSchema] # RAG results from tool library
    past_spec_matches: list[SpecPattern]   # RAG results from past specs
    spec: AgentSpec                        # generated specification
    critique: CritiqueReport              # critic findings
    spec_iteration: int                    # current architect-critic loop count
    spec_approved: bool                    # human checkpoint 2

    # Stage 4-5: Builder + Tester
    generated_code: CodeBundle             # compiled code output
    test_cases: list[TestCase]             # auto-generated from spec contracts
    test_results: TestReport               # execution results
    failure_traces: list[FailureTrace]     # mapped back to spec decisions
    build_iteration: int                   # current build-test loop count

    # Stage 6: Learning
    build_outcome: BuildOutcome            # final record for memory
```

3.2 Graph Definition

```
graph = StateGraph(FrankensteinState)

# Nodes
graph.add_node("elicitor", elicitor_agent)
graph.add_node("human_review_requirements", human_checkpoint_1)
```

```
graph.add_node("architect", architect_agent)
graph.add_node("critic", critic_agent)
graph.add_node("human_review_spec", human_checkpoint_2)
graph.add_node("builder", builder_agent)
graph.add_node("tester", tester_agent)
graph.add_node("learner", learning_agent)

# Edges
graph.add_edge("elicitor", "human_review_requirements")
graph.add_edge("human_review_requirements", "architect")
graph.add_edge("architect", "critic")

# Conditional: Critic → Architect (loop) or → Human Review
graph.add_conditional_edges("critic", route_after_critique)
# if criticals > 0 AND spec_iteration < MAX_SPEC_ITER → "architect"
# else → "human_review_spec"

graph.add_edge("human_review_spec", "builder")
graph.add_edge("builder", "tester")

# Conditional: Tester → Builder (code fix) or → Architect (spec fix) or → Learner (
graph.add_conditional_edges("tester", route_after_test)
# if tests_passed → "learner"
# if failure is code-level AND build_iteration < MAX_BUILD_ITER → "builder"
# if failure is spec-level AND build_iteration < MAX_BUILD_ITER → "architect"
# if max iterations reached → "learner" (with partial success flag)

graph.add_edge("learner", END)
```

3.3 Routing Logic

```
def route_after_critique(state: FrankensteinState) -> str:
    criticals = [f for f in state["critique"].findings if f.severity == "critical"]
    if len(criticals) > 0 and state["spec_iteration"] < MAX_SPEC_ITERATIONS:
        return "architect"  # loop back with critique attached
    return "human_review_spec"

def route_after_test(state: FrankensteinState) -> str:
    if state["test_results"].all_passed:
        return "learner"
```

```
    if state["build_iteration"] >= MAX_BUILD_ITERATIONS:
        return "learner"  # partial success, store learnings anyway
    # Analyze failure level
    for trace in state["failure_traces"]:
        if trace.root_cause_level == "spec":
            return "architect"  # spec-level fix needed
    return "builder"  # code-level fix sufficient
```

4. Internal Agent Engineering

4.1 Elicitor Agent

Implementation: LangGraph subgraph with a loop.

```
analyze_prompt → identify_gaps → generate_questions → receive_answers → update_requ
     ↑
     └──────────────────────────── (if incomplete) ←──────────────────────────────
```

How it decides what to ask:

The Elicitor runs the raw prompt through a gap analysis against a completeness checklist. Each of the five categories (Input/Output, Process, Data, Edge Cases, Quality Bar) has required fields. The Elicitor checks which fields the prompt already answers and generates questions only for gaps.

```
class RequirementsDoc(BaseModel):
    domain: str
    inputs: list[DataSpec]           # name, format, description, example
    outputs: list[DataSpec]          # name, format, description, example
    process_steps: list[ProcessStep] # step_number, description, rules, depends_on
    edge_cases: list[EdgeCase]       # description, expected_handling
    quality_criteria: list[QualityCriterion]  # criterion, validation_method
    constraints: list[str]           # budget, time, technical constraints
```

Completeness check: Each field has a `confidence_score` (0-1). The Elicitor loops until all required fields score above 0.7. It asks a maximum of 3 rounds of questions — if gaps remain after 3 rounds, it flags them as assumptions in the requirements doc and proceeds.

Output: Structured `RequirementsDoc` presented to human. Human approves or corrects.

4.2 Architect Agent

Implementation: Sequential chain with RAG retrieval.

Step-by-step process:

```
1. RAG Query: query Chroma for similar past specs
        ↓
2. Task Decomposition: break requirements into discrete tasks
        ↓
3. Tool Matching: for each task, query Tool Schema Library
        ↓
4. Agent Design: group tasks into agents by cohesion
        ↓
5. Flow Design: analyze dependencies → pick orchestration pattern
        ↓
6. Memory Design: determine what agents need to share
        ↓
7. Compile Spec: assemble all decisions into spec schema
```

Task Decomposition — the Architect reads the `process_steps` from the requirements and identifies discrete computational tasks. Each task gets tagged with: what input it needs, what output it produces, what capability it requires (text extraction, calculation, reasoning, generation, API call).

Tool Matching — for each task's required capability, the Architect queries the Tool Schema Library in Chroma:

```
# Query: "extract text from PDF documents"
# Returns: [
#   {id: "pdf_parser_pymupdf", accepts: ["pdf"], outputs: ["text"], limitations: ["
#   {id: "pdf_parser_unstructured", accepts: ["pdf"], outputs: ["text", "tables"],
#   {id: "ocr_tesseract", accepts: ["image", "scanned_pdf"], outputs: ["text"], lim
# ]
```

The Architect selects based on format compatibility with upstream/downstream tasks. If Task A outputs JSON and Task B needs JSON, the tool must accept JSON. This is explicit matching, not guessing.

Agent Grouping — tasks are grouped into agents based on cohesion (related tasks) and coupling (data dependencies). A task that produces output consumed by another task in a tight loop should be in the same agent. Independent task chains become separate agents.

Flow Design — the Architect builds a dependency graph from task relationships: - No dependencies between agent groups → parallel execution - Linear chain of dependencies → sequential execution - Complex branching with conditions → graph-based (LangGraph) - Simple role delegation → hierarchical (CrewAI)

This determines the framework target: CrewAI for role-based crews, LangGraph for state-dependent flows.

Spec Schema:

```
metadata:
  name: string
  domain: string
  framework_target: crewai | langgraph
  created_from_pattern: string | null  # reference to past spec used as base

agents:
  - id: string
    role: string
    goal: string
    backstory: string
    tools: [tool_id]
    reasoning_strategy: react | cot | plan_execute
    receives_from: [agent_id]
    sends_to: [agent_id]

tools:
  - id: string
    library_ref: string          # reference to Tool Schema Library entry
    config: object               # tool-specific configuration
    accepts: [format]
    outputs: [format]

memory:
  strategy: short_term | long_term | shared | none
  shared_keys: [string]         # what data is shared between agents
  persistence: session | permanent

execution_flow:
  pattern: sequential | parallel | hierarchical | graph
  graph:                        # only if pattern == graph
    nodes: [agent_id]
```

```
    edges:
      - from: agent_id
        to: agent_id
        condition: string | null
        data_contract:
          fields: [field_name]
          format: string

error_handling:
  - agent_id: string
    on_failure: retry | fallback | skip | abort
    max_retries: int
    fallback_agent: string | null

io_contracts:
  - agent_id: string
    input_schema:
      fields: [{name: string, type: string, required: bool}]
    output_schema:
      fields: [{name: string, type: string, required: bool}]
```

4.3 Critic Agent

Implementation: Parallel chain — runs multiple attack vectors simultaneously, aggregates findings.

Attack vectors (each is a separate chain):

Vector	What It Checks	How
Format Compatibility	Does Agent A's output format match Agent B's input format?	Walks every edge in the execution graph, compares `output_schema` → `input_schema`
Tool Validation	Does each tool actually support the data format assigned to it?	Cross-references tool's `accepts` field against the data it will receive
Dead-End Analysis	Can any agent fail without being caught?	Checks every agent's `on_failure` config. Flags any agent with no error handling
Dependency Completeness	Does every agent receive all fields it needs?	Compares each agent's `input_schema.required` fields against upstream `output_schema`

Vector	What It Checks	How
Resource Conflicts	Do parallel agents write to the same shared memory keys?	Checks `memory.shared_keys` against parallel execution paths
Circular Dependencies	Does the execution graph contain cycles?	Runs topological sort on the graph. If it fails, there's a cycle

Output structure:

```
class CritiqueReport(BaseModel):
    findings: list[Finding]
    summary: str
    iteration: int

class Finding(BaseModel):
    vector: str                # which attack vector found this
    severity: Literal["critical", "warning", "suggestion"]
    description: str           # what's wrong
    location: str              # which agent/edge/tool is affected
    evidence: str              # specific spec values that conflict
    suggested_fix: str         # what the Architect should change
```

Loop mechanics: The Critic's findings are attached to state. When routing back to Architect, the Architect receives `state["critique"].findings` and must address every critical finding. The Architect's revised spec is passed to the Critic again. The Critic only checks against previously found issues plus any new issues introduced by the revision.

4.4 Builder Agent

Implementation: Template-based code generation with framework-specific compilers.

Compilation process:

```
Read spec → Select framework compiler → For each agent: generate definition
    → For each tool: generate binding → Generate orchestration code
    → Generate entry point → Assemble project → Validate
```

Project output structure:

```
generated_agent/
├── main.py                # entry point
├── agents.py              # agent definitions (roles, goals, backstories)
├── tools.py               # tool implementations and bindings
├── orchestration.py       # CrewAI crew or LangGraph graph definition
├── config.yaml            # agent and tool configuration
├── requirements.txt       # Python dependencies
└── tests/
    └── test_pipeline.py   # auto-generated test cases from spec contracts
```

CrewAI compiler — maps spec agents to `Agent()` instances, tools to `Tool()` instances, execution flow to `Crew()` with `process=Process.sequential|Process.hierarchical`.

LangGraph compiler — maps spec agents to node functions, execution flow edges to `add_edge()` / `add_conditional_edges()`, state to `TypedDict`, memory to `MemorySaver` or custom state fields.

Validation before execution: 1. Syntax check — `py_compile` on all generated .py files 2. Import check — verify all imports resolve against requirements.txt 3. Schema check — verify generated code matches spec contracts (function signatures match I/O contracts)

4.5 Tester Agent

Implementation: Docker execution engine + output validator.

Test case generation:

The Tester reads the spec's `io_contracts` and generates test cases:

```
# From spec: input is bank_statement.pdf, output is risk_score.json
# Generated test:
class TestCase(BaseModel):
    name: str                    # "test_bank_statement_processing"
    input_data: dict             # sample input (synthetic or from domain templates
    expected_output_schema: dict # JSON schema the output must match
    quality_checks: list[str]    # ["output.risk_score is between 0 and 1",
                                 #  "output.reasoning is non-empty string"]
```

For hackathon scope: test inputs are synthetic (generated by LLM based on domain knowledge from requirements doc). For production: would use real sample data.

Execution flow:

```
1. Build Docker image from generated_agent/ + requirements.txt
2. Run container with test input mounted
3. Capture stdout, stderr, exit code
4. Parse output against expected_output_schema
5. Run quality_checks against parsed output
6. If failure → generate FailureTrace
```

Failure tracing:

```
class FailureTrace(BaseModel):
    test_name: str
    error_type: Literal["crash", "wrong_output", "missing_field", "quality_fail"]
    raw_error: str                # actual error message or output
    failing_agent: str            # which agent in the generated pipeline failed
    root_cause_level: Literal["code", "spec"]
    root_cause_analysis: str      # "Agent 2 received XML but tool expects JSON"
    spec_decision_responsible: str # "spec.tools[1].library_ref = xml_parser"
    suggested_fix: str            # "Change to json_parser or add format converter"
```

The `root_cause_level` determines routing: `"code"` loops back to Builder (fix the generated code), `"spec"` loops back to Architect (fix the design decision).

4.6 Learner Agent

Implementation: Post-build processor that writes structured records to Chroma.

What gets stored after every build:

```
class BuildOutcome(BaseModel):
    requirements_hash: str        # for similarity matching
    requirements_summary: str     # embedded for RAG retrieval
    domain: str
    spec_snapshot: AgentSpec      # the final validated spec
    framework_used: str           # crewai or langgraph
    tools_used: list[str]         # tool IDs from library
    test_results: TestReport
    iterations_needed: int        # how many architect-critic + build-test loops
    failure_patterns: list[str]   # what went wrong and was fixed
    anti_patterns: list[str]      # patterns that caused failures
    success_patterns: list[str]   # patterns that worked well
```

5. Chroma Memory Architecture

5.1 Collections

Collection	Content	Embedding Source	Query Pattern
`tool_schemas`	Validated tool definitions	Tool description + capabilities text	Architect queries with task capability requirements
`spec_patterns`	Past validated specs + outcomes	Requirements summary text	Architect queries with current requirements before generating
`anti_patterns`	Failed patterns with explanations	Failure description text	Architect queries to check "has this pattern failed before?"
`domain_insights`	Domain-specific learnings	Domain + task type text	Elicitor queries to load domain-specific question templates

5.2 Tool Schema Library Structure

Each tool in the library is a structured record:

```
class ToolSchema(BaseModel):
    id: str                       # "pdf_parser_pymupdf"
    name: str                     # "PyMuPDF PDF Parser"
    description: str              # "Extracts text and tables from PDF files"
    category: str                 # "document_extraction"
    accepts: list[str]            # ["pdf"]
    outputs: list[str]            # ["text", "tables"]
    output_format: str            # "json"
    limitations: list[str]        # ["no OCR for scanned documents"]
    dependencies: list[str]       # ["pymupdf>=1.23.0"]
    code_template: str            # actual Python code for this tool
    compatible_with: list[str]    # tool IDs that work well downstream
    incompatible_with: list[str]  # tool IDs known to conflict
```

The `compatible_with` and `incompatible_with` fields are populated by the Learner from past build outcomes. Initially seeded manually, then grows through usage.

5.3 RAG Query Flow

```
Architect receives requirements
    ↓
Query 1: spec_patterns collection
    "Find specs similar to: [requirements_summary]"
    → Returns top-3 past specs with their outcomes
    → Architect uses successful ones as structural reference
    ↓
Query 2: tool_schemas collection
    For each task: "Find tools that can: [task_capability]"
    → Returns ranked tool options with compatibility info
    → Architect selects based on format chain compatibility
    ↓
Query 3: anti_patterns collection
    "Has any pattern similar to [proposed_pattern] failed before?"
    → Returns matching anti-patterns
    → Architect avoids known failure modes
```

6. Technology Stack

6.1 LLM Strategy — Model Per Agent via OpenRouter

All LLM calls route through OpenRouter. Each agent uses a different model optimized for its task. Architect and Critic deliberately use different model families to avoid shared blind spots in adversarial review.

Agent	Model	Reasoning
Elicitor	`gpt-4o-mini`	Fast, good conversational skills for interactive Q&A with human
Architect	`claude-sonnet-4-6`	Strongest structured reasoning for spec generation and architectural decisions
Critic	`gpt-4o`	Different model family from Architect — cross-model adversarial review catches flaws same-model review misses
Builder	`claude-sonnet-4-6`	Strong code generation, deep knowledge of CrewAI/LangGraph patterns
Tester	`gpt-4o-mini`	Error analysis and trace mapping — doesn't need top-tier model
Learner	`gpt-4o-mini`	Lightweight — just structuring data for Chroma storage

6.2 Full Stack

Layer	Technology	Role
LLM Gateway	OpenRouter	Single API for all model access, model-per-agent routing
Pipeline Orchestration	LangGraph	Frankenstein's own internal pipeline — StateGraph with conditional edges for feedback loops
Generated Agent Framework	CrewAI or LangGraph	Architect decides per case — CrewAI for role-based crews, LangGraph for state-dependent flows
Vector Database	Chroma	Tool schemas, past specs, learnings, anti-patterns for RAG retrieval
Code Execution	Docker (pre-built base image)	Base image with Python + CrewAI + LangGraph + common packages pre-installed. Generated code mounted at runtime. Fast startup (~2-3s)
Backend	FastAPI	API server — handles chat sessions, pipeline orchestration, Docker management, Chroma queries
Frontend	React	Chat interface for prompt input, Elicitor Q&A, spec review/approval, agent delivery and download

6.3 Docker Execution Environment

Pre-built base image (`frankenstein-runner`):

```
FROM python:3.11-slim
RUN pip install crewai langgraph langchain-core langchain-openai \
    chromadb pymupdf pandas requests beautifulsoup4 pydantic
WORKDIR /agent
# Generated code mounted here at runtime
CMD ["python", "main.py"]
```

Runtime flow: 1. Builder outputs `generated_agent/` directory 2. FastAPI mounts directory into running container: `docker run -v ./generated_agent:/agent frankenstein-runner` 3. Tester captures stdout/stderr via Docker SDK 4. Container killed after timeout (60s default) 5. Output parsed and validated

7. Datasets

Dataset	Purpose
Tool Schema Library	Curated library of validated tool definitions with input/output format specs, capabilities, and known limitations. Pre-seeded with tools needed for PS-08 and PS-06 demos
Spec Pattern Memory	Growing collection of validated specs, outcomes, and anti-patterns in Chroma, queried via RAG
Domain Question Templates	Per-domain structured question sets for the Elicitor's extraction protocol
Code Templates	Framework-specific (CrewAI / LangGraph) code patterns mapped to spec elements for deterministic compilation

7.1 Pre-Seeded Tool Library (MVP)

Tools needed for demo problem statements:

PS-08 — Loan Underwriting Co-Pilot:

Tool ID	Category	Accepts	Outputs
`pdf_parser_pymupdf`	document_extraction	pdf	text, tables
`ocr_tesseract`	document_extraction	image, scanned_pdf	text
`financial_calculator`	computation	json (numbers)	json (ratios, scores)
`rule_engine`	reasoning	json (data + rules)	json (pass/fail + reasoning)
`report_generator`	generation	json (analysis)	markdown, pdf
`web_search`	research	query string	json (results)

PS-06 — Supplier Reliability Scoring Agent:

Tool ID	Category	Accepts	Outputs
`csv_parser`	data_ingestion	csv	json (structured data)
`statistical_analyzer`	computation	json (numerical data)	json (statistics, trends)
`scoring_engine`	reasoning	json (metrics + weights)	json (scores + reasoning)
`data_visualizer`	generation	json (data)	image (charts), html

Tool ID	Category	Accepts	Outputs
`report_generator`	generation	json (analysis)	markdown, pdf

General-Purpose Tools (available to all builds):

Tool ID	Category	Accepts	Outputs
`web_search`	research	query string	json (results)
`file_reader`	data_ingestion	txt, json, yaml	text, json
`json_transformer`	data_processing	json	json (restructured)
`llm_reasoner`	reasoning	text (prompt)	text (response)
`code_executor`	computation	python code string	execution output

8. What Frankenstein Does Not Do

Clarity on scope:

- **Does not understand domains autonomously** — relies on the human for domain knowledge. Frankenstein is the engineering team, not the product manager.
- **Does not replace prompt engineering entirely** — the Elicitor improves input quality, but garbage domain knowledge in still produces garbage agents out.
- **Does not guarantee perfect agents on first pass** — the test-and-fix loop exists because first builds will have issues. The system's strength is autonomous correction, not perfection.
- **Does not work without a Tool Schema Library** — the Architect can only select tools it knows about. An empty library means no useful output. The library must be pre-built and maintained.